# Software Requirements Specification

for

# MediConnect — Doctor Appointment System

Prepared by

Towhidul Islam - 2212172042
Subaita Maryam - 2222743642
Md. Al-Amin - 2131585642
Atika Sharmin – 2412562642
SADMAN SAMI - 2231223642

**North South University**
**Software Engineering (CSE - 327)**
**7 July 2026**

# Contents

# Revision History

| Revision | Date | Author(s) |
|---|---|---|
| 1.0 | 1.07.2026 | Towhidul Islam |
| 2.0 | 2.07.2026 | Subaita Maryam |
| 3.0 | 3.07.2026 | Md. Al-Amin |
| 4.0 | 4.07.2026 | Atika Sharmin |
| 5.0 | 5.07.2026 | SADMAN SAMI |

# Chapter 1 Introduction

Access to timely medical consultation has become one of the most critical challenges in modern healthcare, particularly in densely populated countries such as Bangladesh, where hospital queues, limited appointment slots, and fragmented patient records often delay proper treatment. Traditional appointment booking- done through phone calls, walk-ins, or paper registers - wastes valuable time for both patients and physicians, and offers no unified place for individuals to keep track of their consultations, prescriptions, and medical history. A dependable digital platform that connects patients directly with verified doctors, records every interaction, and preserves it in a structured way can substantially improve the healthcare experience and reduce the operational burden on clinics.

## 1.1 Purpose

The purpose of this project is to build MediConnect, a web-based Doctor Appointment System that allows patients to discover certified doctors by specialization, view weekly availability, book consultation slots, receive digital prescriptions after their visits, and maintain a personal record of past appointments and medical conditions. The platform also gives doctors a workspace to publish their profile, publish weekly availability, review incoming booking requests, approve or reject them, and issue prescriptions once a consultation is completed. An administrative role sits above both, responsible for verifying newly registered doctors, curating the department catalog, moderating public feedback, and monitoring high-level system statistics. Together, these three roles form a single connected workflow around one central record - the appointment - that every feature revolves around.

## 1.2 Intended Audience

- Developers
- Project Testers
- Project Supervisor and Evaluators
- Marketing and Deployment Team

## 1.3 Intended Use

**Developers:**

Developers refer to this document to understand the complete feature scope of MediConnect before writing any code. It clarifies which modules exist, how the three user roles interact, what the system is expected to enforce, and where flexibility is allowed. During implementation, it serves as a checklist to ensure that every declared functional requirement has a corresponding view, form, and template. During later iterations or maintenance, this document acts as the reference against which any proposed change is compared - features can be revised, extended, or optimized only with an updated view of what was originally agreed upon.

**Project Testers:**

Testers use this specification to design test cases that verify each functional requirement. Because the document lists user actions as small, self-contained stories with confirmation criteria, testers can convert each item directly into a test scenario - both positive (the feature behaves as described) and negative (invalid inputs, unauthorized access, and edge conditions are rejected gracefully). The non-functional requirements section additionally guides performance testing, browser-compatibility checks, and access-control validation.

**Project Supervisor and Evaluators:**

The academic supervisor and evaluation committee use the SRS as the primary document to judge whether the delivered software meets its declared scope. It gives evaluators a fixed reference point for grading completeness, correctness, and quality of engineering work independently from the running application.

**Marketing and Deployment Team:**

When MediConnect is prepared for public release or presented in a portfolio, the marketing and deployment team relies on this document to communicate the system's capabilities accurately - without over-promising features that are not yet implemented or under-representing what already exists. It helps produce accurate landing-page copy, feature checklists, and demo scripts.

## 1.4 Product Scope

MediConnect is a brand-new, web-based application built entirely from scratch, with no dependency on any existing appointment platform's codebase. It is a browser-first system, responsive from small mobile screens up to desktop displays, and delivers a familiar dashboard experience for each of its three user roles. The following are the concrete benefits the system provides.

**Benefits of the application:**

1. Patients can search for doctors by name, specialization, or department.
2. Patients can view a doctor's public profile, including qualifications, consultation fee, average patient rating, and weekly availability.
3. Patients can request an appointment on a chosen date and time, describing the reason for the visit.
4. Patients can track the status of every appointment they have ever made, and cancel one before it is completed.
5. Patients can read prescriptions issued to them digitally, and print them if they wish.
6. Patients can maintain a personal medical history log of pre-existing conditions and diagnoses.
7. Patients can rate and review a doctor after a completed consultation, contributing to a public reputation score.
8. Doctors can publish a professional profile with department, qualifications, experience, and consultation fee.
9. Doctors can define a weekly availability schedule that patients see before booking.
10. Doctors can approve, reject, or mark completed each appointment request they receive.
11. Doctors can write and store a prescription for every completed consultation.
12. Doctors can toggle themselves temporarily unavailable without losing their account or history.
13. Administrators can review, approve, or reject each new doctor registration before that doctor is allowed to log in.
14. Administrators can create, edit, and remove medical departments from the catalog.
15. Administrators can monitor the entire system through a dashboard summarizing users, doctors, patients, appointments, and pending items.
16. Any visitor, whether logged in or not, can submit feedback or a support message through the public contact form.
17. Every logged-in user receives dashboard notifications when a relevant event affects them.

**Objectives:**

1. To provide a reliable and organized digital channel between patients and verified medical professionals.
2. To reduce the effort and waiting time involved in booking a doctor's appointment.
3. To keep a permanent, searchable record of every patient's consultations, prescriptions, and medical background.
4. To give administrators a clear oversight of who is providing medical services on the platform.

**Goals:**

1. To ensure no patient has to physically stand in queue simply to obtain an appointment slot.
2. To make sure every doctor listed on the platform has been individually verified before receiving any bookings.
3. To keep the user experience consistent across mobile, tablet, and desktop devices.
4. To structure the codebase in a way that any future feature - payment, video consultation, mobile app - can be added without redesigning the core.

## 1.5 Risk Definition

1. An unreliable internet connection on the patient's or doctor's device may interrupt an active session, causing the current form submission to fail and requiring the user to re-enter data.
2. A malicious registered user may attempt to book appointments at implausible times (very early morning, past-dated slots) or repeatedly submit spam booking requests, degrading experience for legitimate users.
3. A sudden spike in traffic - for instance during a health advisory or an outbreak - may exceed the capacity of the deployment server, causing slow response times or temporary unavailability.
4. An unapproved doctor may attempt to gain access to the doctor dashboard by circumventing the administrator's approval workflow.
5. A patient may accidentally share their login credentials, exposing their prescriptions and medical history to another person.
6. A dependency library used by the project (Django, Bootstrap, Pillow) may release a critical security update that requires timely re-deployment.

# Chapter 2 Overall Description

MediConnect is a completely new, standalone web application - not a rebuild or fork of any existing system - built to serve as a common ground for three types of users: patients who need medical consultation, doctors who provide it, and administrators who supervise the platform's participants. The application is designed to be responsive on all mainstream web browsers and does not require any additional client-side installation. All persistent data - user profiles, appointments, prescriptions, medical histories, reviews, and notifications - is stored inside the platform's database and served through Django's request/response cycle. This chapter describes the overall picture of who uses the system, in which environment it operates, and under what constraints and assumptions it has been designed.

## 2.1 User Classes and Characteristics

MediConnect distinguishes three well-defined user classes. Every registered account belongs to exactly one class, and the interface, capabilities, and permitted actions each class sees are entirely determined by that classification.

- **Patient:** Any individual seeking a doctor's consultation for themselves or on behalf of another person. Patients register directly through the public sign-up form and gain immediate access. They are the primary users of the search, booking, prescription-viewing, and review features.
- **Doctor:** A licensed medical professional who wishes to accept online consultation requests through the platform. Doctors register through the same sign-up form as patients, but remain locked out of the system until an administrator has manually approved their account. Once approved, they gain a workspace for schedule management, appointment triage, and prescription writing.
- **Administrator:** A privileged operator responsible for overseeing the platform. Administrators are not created through the public registration form and cannot be self-selected by any visitor. They exist only through the project's command-line superuser creation or by an existing administrator granting the role through the internal management panel.

## 2.2 User Needs

Patients need a fast, low-friction way to look up medical professionals without having to phone individual clinics one by one. They need to know, before making a request, whether a doctor is currently accepting patients, what specialization the doctor holds, what other patients have said about them, and when the doctor is available. After the visit, they need permanent access to any prescription they were given, and they need to review the doctor to help future patients choose. On top of all this, patients keep a self-authored medical history log - a running note of chronic conditions, previous diagnoses, and observations that may be relevant to any future doctor they consult.

Doctors need a controlled queue of incoming booking requests so they can approve those that fit their schedule and politely reject those that do not, without being flooded by every request in real time. They need a place to publish their qualifications, define recurring weekly slots, and keep the fee they charge up to date. Once a consultation is over, they need a simple form to record the diagnosis, prescribed medicines, additional advice, and a follow-up date if required. Finally, doctors need to see the reviews patients leave about them, so they understand how their practice is being perceived.

Administrators need continuous visibility into who is joining the platform and what activity is happening on it. Their most time-sensitive task is reviewing new doctor registrations, since every unapproved doctor is effectively locked out. Beyond that, they need to keep the department catalog aligned with the specializations the platform actually serves, moderate the public feedback inbox, and check the health of the system through a summary dashboard.

## 2.3 Operating Environment

The operating environment for MediConnect is described below.

- **Client Operating System:** Any modern operating system that supports a compliant web browser - Windows 10 or newer, macOS, Linux distributions, Android, and iOS.
- **Supported Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, in their two most recent versions.
- **Server-Side Language:** Python 3.10 or newer.
- **Web Framework:** Django 4.2 (long-term support release).
- **Database:** SQLite for local development; PostgreSQL as an option for production deployment (the ORM abstraction means no application code needs to change).
- **Frontend Stack:** HTML5, CSS3, JavaScript, and Bootstrap 5 for responsive styling.
- **Web Server:** Django's built-in development server locally; a WSGI-compatible server such as Gunicorn behind Nginx in production.

## 2.4 Constraints

- The application must be developed in Python using the Django framework, following the Model-View-Template pattern.
- The user interface must be responsive and functional on all mainstream mobile, tablet, and desktop screen sizes.
- The project must be completed within the semester timeline allotted for the CSE 327 Software Engineering course.
- All passwords must be stored using Django's built-in one-way password-hashing mechanism; plaintext storage is not permitted anywhere.
- Any destructive action (account deletion, appointment cancellation, department deletion) must be reversible only through an explicit administrative re-entry - no undo feature is provided in the current scope.
- The system does not integrate with any external payment gateway, video-conferencing service, SMS gateway, or artificial-intelligence service in this release.

## 2.5 Assumptions

- Every user - patient, doctor, or administrator - is comfortable reading and writing English at least at a functional level.
- Every user has access to a device capable of rendering a modern web page (smartphone, tablet, laptop, or desktop) and a working internet connection.
- Doctors who register on the platform hold a valid medical qualification; the administrator's approval workflow is not a substitute for offline license verification and assumes such verification has occurred outside the platform.
- Patients honestly report the reason for the consultation and their own medical history when using the system.
- The server hosting the application maintains a stable connection to the internet with adequate bandwidth to serve concurrent users.

# Chapter 3 Requirements

## 3.1 Functional Requirements

**1. As a patient**

*I want to register for a new account on MediConnect*
*so that I can start booking appointments and managing my health records.*

***Confirmation***

- The user provides a username, full name, email address, phone number, and a password to be confirmed twice.
- The email address must not already be in use by another account.
- The password must meet the minimum strength rules set by the framework.
- Upon successful submission, a patient account and its associated profile are created and the user is directed to the login page.
- The user can log in immediately without any approval step.

**2. As a doctor**

*I want to register for a doctor account*
*so that I can begin providing consultations after verification.*

***Confirmation***

- The doctor completes the same registration form as a patient but selects the "Doctor" role.
- Upon submission, a doctor profile is created but remains in an unapproved state.
- An informational message notifies the doctor that the account is pending administrator approval.
- Any login attempt made before approval is refused with a clear message explaining the reason.

**3. As a registered user**

*I want to log in and log out securely*
*so that only I can access my dashboard and personal records.*

***Confirmation***

- The user enters a username and password on the login page.
- Incorrect credentials produce a clear, non-specific error message that does not reveal which field was wrong.
- On successful login, the user is automatically redirected to the dashboard corresponding to their role.
- The logout action ends the current session immediately and returns the user to the public homepage.

**4. As a registered user**

*I want to reset my forgotten password via email*
*so that I can recover access without needing to contact an administrator.*

***Confirmation***

- The user enters the email address associated with the account on the "Forgot Password" page.
- If the address matches a registered account, an email containing a one-time reset link is dispatched.
- Following the reset link opens a form for choosing a new password, subject to the same strength rules as registration.

- Once the new password is set, the user can log in normally with it.
- Reset links expire after a short interval to limit exposure.

**5. As a registered user**

*I want to update my profile information and change my password from inside the system*
*so that I can keep my details current without having to re-register.*

***Confirmation***

- The profile page shows the user's current first name, last name, email, phone number, and profile picture.
- Any of these fields can be edited and saved.
- The password-change form asks for the current password and the new password twice; a mismatch or wrong current password blocks the change.
- Patients and doctors can, in addition, edit their role-specific fields (age, gender, and blood group for patients; department, specialization, qualification, experience, fee, and biography for doctors).

**6. As a patient**

*I want to search for a doctor by name, specialization, or department*
*so that I can find a professional suited to my need.*

***Confirmation***

- The public doctor directory displays a card for every approved and currently available doctor.
- A search input filters cards by matching the query against doctor name and specialization text.
- A department dropdown filters cards to a single specialization category.
- Only doctors who are both administrator-approved and marked available by themselves appear in the results.
- An empty search state displays a friendly "no results" message rather than a blank page.

**7. As a patient**

*I want to view a doctor's complete public profile*
*so that I can make an informed choice before booking.*

***Confirmation***

- The profile page displays the doctor's name, department, specialization, qualification, years of experience, consultation fee, and biography.
- The doctor's weekly availability schedule is shown as a day-by-day breakdown of start and end times.
- The average patient rating (out of five stars) and the total number of reviews are displayed prominently.
- The ten most recent patient reviews are listed below the profile.
- Logged-in patients see a "Book Appointment" button; unauthenticated visitors are prompted to log in first.

**8. As a patient**

*I want to book an appointment with a selected doctor*
*so that I can secure a consultation slot.*

***Confirmation***

- The booking form asks for a preferred date, a preferred time, and a short description of the reason or symptoms.
- Dates in the past are rejected during form validation.

- A submitted booking is stored with the status "Pending" and the doctor receives an immediate dashboard notification.
- The patient is taken to their appointments list and sees the new request there.

**9. As a patient**

*I want to review all my past and upcoming appointments in one place*
*so that I can keep track of my medical activity.*

***Confirmation***

- The appointments list shows every appointment the patient has ever created, sorted by most recent first.
- Each row displays the doctor's name, date, time, reason, and current status.
- A colored badge distinguishes each status at a glance.
- Rows for completed appointments carry a link to the associated prescription, when one exists.

**10. As a patient**

*I want to cancel an appointment that has not yet taken place*
*so that I can free up the slot when my plans change.*

***Confirmation***

- The cancel option is offered only when the appointment status is "Pending" or "Approved".
- A confirmation prompt appears before the cancellation is finalized.
- Cancelling changes the status to "Cancelled" and dispatches a notification to the doctor.
- An appointment that has already been completed, rejected, or cancelled cannot be cancelled again.

**11. As a patient**

*I want to view and download the prescriptions issued to me*
*so that I have a permanent digital record of my treatment.*

***Confirmation***

- The prescriptions page lists every prescription associated with the patient's completed appointments.
- Each entry shows the issuing doctor, the appointment date, and a short diagnosis summary.
- Opening a prescription reveals the full diagnosis, prescribed medicines, additional advice, and follow-up date.
- A print button produces a clean paper-friendly copy of the prescription.
- No other patient can access this prescription; access is strictly limited to the associated patient, the issuing doctor, and administrators.

**12. As a patient**

*I want to maintain a personal medical history log*
*so that I can share past conditions with any doctor I consult in the future.*

***Confirmation***

- The medical history page shows a chronological list of conditions the patient has recorded.
- Each entry captures a condition name, the date of diagnosis, and free-text notes.
- The patient can add new entries or delete existing ones at any time.
- Only the owner sees their history; no doctor or other patient can access it.

**13. As a patient**

*I want to leave a rating and comment for a doctor after a completed consultation*
*so that I can share my experience and help other patients decide.*

***Confirmation***

- The review form is only accessible after the patient has at least one completed appointment with the given doctor.
- The form accepts a rating from one to five stars and an optional written comment.
- Each patient can leave exactly one review per doctor; submitting again updates the same review rather than creating a duplicate.
- The submitted review contributes immediately to the doctor's public average rating.

**14. As a doctor**

*I want to manage my professional profile and weekly availability*
*so that patients see accurate information before booking.*

***Confirmation***

- The doctor can update department, specialization, qualification, years of experience, consultation fee, and biography from the profile page.
- An "Available" toggle lets the doctor become temporarily invisible in the public directory without deleting the account.
- The availability page lets the doctor add or remove recurring weekly slots, each defined by a weekday, start time, and end time.
- Attempting to add a slot whose end time is not after its start time is rejected with a validation message.
- Attempting to add a slot identical to an existing one produces a friendly "slot already exists" message.

**15. As a doctor**

*I want to review and act on the appointment requests I receive*
*so that I can manage my consultation queue.*

***Confirmation***

- The appointments management page lists every appointment addressed to the doctor, filterable by status.
- Pending appointments offer "Approve" and "Reject" actions.
- Approved appointments offer a "Mark Complete" action.
- Completed appointments offer a "Write Prescription" action if none has been written yet.
- Every action dispatches a notification to the patient describing the outcome.

**16. As a doctor**

*I want to write a prescription for a completed consultation*
*so that my patient has a digital record to follow.*

***Confirmation***

- The prescription form becomes available only after the appointment has been marked completed.
- The form accepts a diagnosis, a list of medicines (one per line), optional advice, and an optional follow-up date.
- Each completed appointment can carry exactly one prescription.
- Submitting the form saves the prescription and sends a notification to the patient with a direct link to it.

**17. As a doctor**

*I want to see the reviews left by my patients*
*so that I understand how my consultations are being received.*

***Confirmation***

- The reviews page shows every review written about the doctor, newest first.
- Each entry displays the patient's name, star rating, comment, and submission date.
- The current average rating and total review count are shown above the list.
- Reviews are read-only from the doctor's side; no reply or edit action is available in this release.

**18. As an administrator**

*I want to review pending doctor registrations and approve or reject each one*
*so that only verified professionals gain access to the platform.*

***Confirmation***

- The doctor management page lists every doctor account, with filters for "Pending" and "Approved" states.
- Approving an account changes its status to approved and sends a confirmation notification to the doctor.
- The doctor cannot log in until this approval has taken place.
- Rejecting a pending account removes it from the system.
- Every state-changing action prompts for confirmation before it is finalized.

**19. As an administrator**

*I want to manage the catalog of medical departments*
*so that doctors can be categorized correctly and patients can filter their searches.*

***Confirmation***

- The department page lists every department with its name, description, and icon reference.
- The administrator can add a new department, edit an existing one, or delete one.
- Deleting a department does not remove the doctors previously assigned to it; those doctors simply become unassigned and can be edited to pick a new department.
- Department names must be unique across the catalog.

**20. As an administrator**

*I want to view a summary dashboard of the whole platform*
*so that I can monitor overall system health and activity at a glance.*

***Confirmation***

- The dashboard shows the total number of users, approved doctors, pending doctors, patients, departments, and appointments.
- Appointment counts are further split by status.
- The five most recently created appointments and the five most recent feedback messages appear beneath the summary numbers.
- A dedicated reports page breaks the same data down by department and by appointment status.

**21. As a visitor**

*I want to send a message through a public contact form*
*so that I can ask a question or share feedback without registering.*

***Confirmation***

- The contact form is accessible to anyone, whether logged in or not.
- The visitor supplies a name, email, subject, and message body.
- Successful submission produces a confirmation message on the same page.
- Logged-in visitors see their name and email pre-filled in the form for convenience.
- Every submission is stored and appears in the administrator's feedback inbox.

**22. As a logged-in user**

*I want to see dashboard notifications for events that concern me*
*so that I never miss an important status change.*

***Confirmation***

- A bell icon in the top navigation bar shows a numeric badge with the count of unread notifications.
- Clicking the bell reveals the five most recent notifications with links to the relevant page.
- A dedicated "Notifications" page lists the full history and marks every entry as read on view.
- Notifications are role-specific: doctors receive booking, cancellation, and approval events; patients receive status-change and prescription events; administrators do not receive notifications.

## 3.2 Non Functional Requirements

***Performance Requirements***

- Any page must render within three seconds under normal network conditions.
- The system must sustain at least fifty simultaneous logged-in users on a single application server without noticeable degradation.
- Database queries on pages listing multiple items (doctor directory, appointment lists, notifications) must be optimized to avoid one-query-per-row patterns.

***Security Requirements***

- All passwords must be stored as one-way hashes using the framework's default hashing algorithm; plaintext storage is not permitted.
- Every form submission must be protected against Cross-Site Request Forgery through the framework's CSRF token mechanism.
- Access to role-specific dashboards and actions must be enforced on the server side; hiding a button in the interface is not sufficient.
- A patient must not be able to view another patient's prescriptions or medical history under any circumstance.
- A doctor must not be able to see appointments addressed to another doctor.
- Session cookies must be marked HTTP-only to prevent access from client-side scripts.

***Usability Requirements***

- The interface must be operable on screens as small as three hundred and twenty pixels wide.
- Every form must display specific, understandable error messages next to the offending field.
- Every destructive action must ask for confirmation before it is finalized.
- The color palette and iconography must remain consistent across all pages of the application.

***Reliability Requirements***

- The application must not lose any data submitted through a valid form during normal operation.
- Server errors must be logged in a way that allows a developer to diagnose the cause without exposing internal details to the end user.
- A restart of the application server must not require any manual data reconstruction.

***Maintainability Requirements***

- The codebase must be organized into small, single-purpose Django applications, each responsible for one feature area.
- Business rules that are checked in more than one place must be extracted into a reusable helper or model method.
- Adding a new field to a model must not require touching more than the model, its form, and one or two templates.

***Portability Requirements***

- The project must run on Windows, macOS, and Linux without any operating-system-specific modification.
- Switching from SQLite to PostgreSQL must require only a configuration change, not any code change.

***Error Handling Requirements***

- Unexpected exceptions must not leak stack traces to the end user in production mode.
- Client-side validation failures must be echoed as friendly messages that describe how to correct the input.
- Attempts to access a non-existent record must produce a clean "Not Found" page rather than a raw error.

***Safety Requirements***

- The application must not encourage any medical decision that would normally require an in-person consultation, and must display an appropriate disclaimer on relevant pages.
- Prescription data must be preserved in an unaltered form once written; edits or deletions of an issued prescription are not permitted in the current release.

# Appendices

## Appendix A  Glossary

**SRS:** A Software Requirements Specification is a document that describes what a software system will do and how it will be expected to perform. It lists functional and non-functional requirements, and often includes user stories that clarify how each requirement will be used in practice.

**Django:** A high-level open-source Python web framework that encourages rapid development and follows the Model-View-Template pattern. Django provides an object-relational mapper, a template engine, a routing system, and a built-in administrative interface.

**MVT:** Model-View-Template, the architectural pattern used by Django. The Model describes the data schema, the View contains request-handling logic, and the Template renders the HTML output.

**ORM:** Object-Relational Mapper. A translation layer that allows a program to interact with a relational database using programming-language objects rather than raw SQL statements.

**SQLite:** A self-contained, file-based relational database engine used as the default data store during MediConnect's development phase.

**PostgreSQL:** A powerful, open-source relational database system suitable for production deployment of Django-based applications.

**Bootstrap 5:** A widely used open-source frontend framework that provides responsive layout utilities and pre-styled interface components used throughout MediConnect.

**CRUD:** Create, Read, Update, and Delete - the four basic operations performed on persistent data records.

**CSRF:** Cross-Site Request Forgery. An attack in which a malicious website causes a logged-in user's browser to submit an unwanted request to a different site. Django's CSRF token mechanism defends against this attack.

**Session:** A server-side record that identifies a logged-in user across multiple page requests, typically referenced through a cookie stored in the user's browser.

**Notification:** A short in-application message written to a specific user's inbox when an event affecting them occurs. In MediConnect, notifications appear as a numeric badge on the top navigation bar.

**Appointment:** The central record of MediConnect that links one patient to one doctor at a specific date and time, and passes through the statuses Pending, Approved, Rejected, Completed, or Cancelled during its lifecycle.

**Prescription:** A digital document issued by a doctor after a completed consultation, containing the diagnosis, list of medicines, additional advice, and an optional follow-up date. Every prescription is linked to exactly one completed appointment.

**Availability:** A recurring weekly time slot, defined by a weekday together with a start and end time, during which a doctor is willing to accept appointments.

**Approval:** The administrative act of allowing a newly registered doctor account to log in and start operating on the platform. Until approval, a doctor's account exists in a locked state.

**Dashboard:** The role-specific landing page that each logged-in user is directed to after signing in, summarizing the most relevant information for that role at a glance.

**Review:** A patient's public one-to-five-star rating and optional written comment about a doctor, submitted only after at least one completed consultation between the two.

**Bcrypt / PBKDF2:** One-way password-hashing algorithms used by Django to store user passwords in a form that cannot be reversed to reveal the original text.